

Correct
 What is a three letter word for mouse trap? **cat**
 Correct
 You got 66 % right out of 3

10.0.4 Exercises

Expand the test.py program so it has a menu giving the option of taking the test, viewing the list of questions and answers, and an option to quit. Also, add a new question to ask, "What noise does a truly advanced machine make?" with the answer of "ping".

Solution

Expand the test.py program so it has menu giving the option of taking the test, viewing the list of questions and answers, and an option to quit. Also, add a new question to ask, "What noise does a truly advanced machine make?" with the answer of "ping".

```
## This program runs a test of knowledge

questions = [["What color is the daytime sky on a clear day? ", "blue"],
             ["What is the answer to life, the universe and everything? ",
              "42"],
             ["What is a three letter word for mouse trap? ", "cat"],
             ["What noise does a truly advanced machine make?", "ping"]]

# This will test a single question
# it takes a single question in
# it returns True if the user typed the correct answer, otherwise False

def check_question(question_and_answer):
    # extract the question and the answer from the list
    question = question_and_answer[0]
    answer = question_and_answer[1]
    # give the question to the user
    given_answer = input(question)
    # compare the user's answer to the testers answer
    if answer == given_answer:
        print("Correct")
        return True
    else:
        print("Incorrect, correct was:", answer)
        return False

# This will run through all the questions

def run_test(questions):

    if len(questions) == 0:
        print("No questions were given.")
        # the return exits the function
        return
    index = 0
    right = 0
    while index < len(questions):
        # Check the question
        if check_question(questions[index]):
            right = right + 1
        # go to the next question
        index = index + 1
```

```
        index = index + 1
    # notice the order of the computation, first multiply, then divide
    print("You got", right * 100 / len(questions),
          "% right out of", len(questions))

#showing a list of questions and answers
def showquestions():
    q = 0
    while q < len(questions):
        a = 0
        print("Q:" , questions[q][a])
        a = 1
        print("A:" , questions[q][a])
        q = q + 1

# now let's define the menu function
def menu():
    print("-----")
    print("Menu:")
    print("1 - Take the test")
    print("2 - View a list of questions and answers")
    print("3 - View the menu")
    print("5 - Quit")
    print("-----")

choice = "3"
while choice != "5":
    if choice == "1":
        run_test(questions)
    elif choice == "2":
        showquestions()
    elif choice == "3":
        menu()
    print()
    choice = input("Choose your option from the menu above: ")
```

11 For Loops

With a for loop, we can repeat a piece of code. Let's look at an example:

```
one_to_ten = range(1, 11)
for count in one_to_ten:
    print(count)
```

The output of this code looks as follows:

```
1
2
3
4
5
6
7
8
9
10
```

The output looks awfully familiar but the program code looks different. The first line uses the `range` function. The `range` function uses two arguments like this `range(start, finish)`. `start` is the first number that is produced. `finish` is one larger than the last number. Note that this program could have been done in a shorter way:

```
for count in range(1, 11):
    print(count)
```

The `range` function returns an iterable. This can be converted into a list with the `list` function. which will then be the dominant number. Here are some examples to show what happens with the `range` command:

```
>>> range(1, 10)
range(1, 10)
>> list(range(1, 10))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>> list(range(-32, -20))
[-32, -31, -30, -29, -28, -27, -26, -25, -24, -23, -22, -21]
>> list(range(5,21))
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
>> list(range(5))
[0, 1, 2, 3, 4]
>> list(range(21, 5))
[]
```

The next line `for count in one_to_ten:` uses the `for` control structure. A `for` control structure looks like `for variable in list:`. `list` is gone through starting with the first element of the list and going to the last. As `for` goes through each element in a list it puts

each into **variable**. That allows **variable** to be used in each successive time the **for** loop is run through. Here is another example (you don't have to type this) to demonstrate:

```
demolist = ['life', 42, 'the universe', 6, 'and', 7, 'everything']
for item in demolist:
    print("The current item is:",item)
```

The output is:

```
The current item is: life
The current item is: 42
The current item is: the universe
The current item is: 6
The current item is: and
The current item is: 7
The current item is: everything
```

Notice how the **for** loop goes through and sets **item** to each element in the list. So, what is **for** good for? The first use is to go through all the elements of a list and do something with each of them. Here's a quick way to add up all the elements:

```
list = [2, 4, 6, 8]
sum = 0
for num in list:
    sum = sum + num

print("The sum is:", sum)
```

with the output simply being:

```
The sum is: 20
```

Or you could write a program to find out if there are any duplicates in a list like this program does:

```
list = [4, 5, 7, 8, 9, 1, 0, 7, 10]
list.sort()
prev = None
for item in list:
    if prev == item:
        print("Duplicate of", prev, "found")
    prev = item
```

and for good measure:

```
Duplicate of 7 found
```

Okay, so how does it work? Here is a special debugging version to help you understand (you don't need to type this in):

```
l = [4, 5, 7, 8, 9, 1, 0, 7, 10]
print("l = [4, 5, 7, 8, 9, 1, 0, 7, 10]", "\t\tl:", l)
l.sort()
print("l.sort()", "\t\tl:", l)
```